

# Ref Returns and Ref Locals in C# 7

In this article, I am going to discuss **How to use Ref Returns and Ref Locals in C# 7** with examples. These two new features are introduced as part of C# 7. Please read our previous article where we discussed [How to Work with Local Functions in C# 7](#) with Examples. As of now, we have used **ref** as the method parameter. But from C# 7, now we can use the **ref** keyword as the return type of method as well as we can also create local variables using the **ref** keyword. Before understanding these two new features (**Ref Returns and Ref Locals in C#**) let's have a look at how we can pass the data and reference as of now.

1. **Passing by Value**
2. **Passing by Reference**
3. **Out parameter**

Let's discuss these concepts first by taking some simple examples before understanding the **ref** returns and **ref** locals in **c#**.

## Passing by Value in C#

If we declare a parameter of a value type, then the value is allocated within the scope of the method and destroyed immediately at the end of the method execution. Thus, the changes to the value within the invoked method will not affect the caller of the method.

## Example to Understand Pass by Value in C#

Let us understand Pass by Value Mechanism in C# with an Example. Please have a look at the below example. In the below example, the method **PassByValue(int Number)** receives a copy of a value type. The variable **"Number"** is allocated within the method. While invoking this method we passed a copy of the variable **"Num"**, not the actual variable. So the change to **"Number"** only happens within the scope of the **PassByValue()** method which will not affect the caller of the method i.e. Main method.

```
using System;
namespace RefReturnsAndRefLocalsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Num = 1;
            //Calling the PassByValue Method using Pass By Value Mechanism
            PassByValue(Num);
            Console.WriteLine($"After the invocation of the {nameof(PassByValue)} method, the value of {nameof(Num)} = {Num}");

            Console.ReadLine();
        }

        //Here, Number Variable will have its own memory address space
        static void PassByValue(int Number)
        {
            //This method will not affect the caller of the method i.e. Main method
        }
    }
}
```

```

        //Changing Number Variable will not chnage Num Variable
        Number = 2;
    }
}
}

```

**Output: After the invocation of the PassByValue method, Num = 1**

As you can see in the above image, it will print the value as 1 because a copy of the value is passed, the value of "Num" never changes, and thus after the invocation of the PassByValue() method, the variable "Num" remains unchanged.

## Passing by Reference in C#

If you want to return the changed data from a method, then you need to use a return type for that method. But in C#, there are some other ways to return values from a method. One of the ways is to declare the method parameter with the **ref** keyword. This allows for returning not only one value but multiple values from a method. With the **ref** parameters in C#, the method can receive and return a value.

### Example to Understand Pass by Reference in C#

In the below example, the parameter **Number** of the **PassByReference()** method has been assigned with the **ref** modifier. This "**ref**" modifier in C# indicates that the method invocation is going to happen by using a pointer. In the below example when the **PassByReference()** method is called the incoming parameter "Number" has the same memory address as the variable "Num" which is passed as the argument that's why modifying the value of "Number" would reflect the changes in the variable "Num" as well. In this case, a new memory location is not created for the method parameter "Number". Here, you can say, "Number" is just an alias of "Num" or an alternative name of "Num". So, if we do any changes using the variable "Number", it will also reflect in the variable "Num". The following example code is self-explained so, please go through the comment lines for a better understanding.

```

using System;
namespace RefReturnsAndRefLocalsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Num = 1;
            //Calling the PassByReference Method using Pass By Reference Mechanism
            PassByReference(ref Num);
            Console.WriteLine($"After the invocation of the {nameof(PassByReference)}
method, {nameof(Num)} = {Num}");

            Console.ReadLine();
        }
        //Declaring Method Parameter with ref keyword
        //Here, Number will share the same memory address as Num
        static void PassByReference(ref int Number)
        {

```

```

        //Changing Number will change the memory address value which is cr
        Num
        {
            Number = 2;
        }
    }
}

```

**Output: After the invocation of the PassByReference method, Num = 2**

With the change, the result on the output console is after the invocation of PassByReference, Num= 2. The point that you need to remember is in the case of ref, the variable must be initialized before sending it to the calling method.

## Out Variable in C#:

As we already discussed with the help of the **ref** modifier in C#, a value can be passed to a method, and also the value can be returned from the method. But, if we have a scenario where we need to only return the value from the method, then in such scenarios we need to use the **out** modifier in C#.

## Example to Understand Out Variable in C#

Let us understand the Out Parameter in C# with an Example. Please have a look at the below example. In the below example, in order to call the Out Method, the variable Num that is passed as a parameter to the Out Method does not need to be initialized. It is initialized within the OUT Method. Here, initialization is optional, as it is mandatory to be initialized in the OUT Method body, else you will get a compile-time error. This is another way of Implementing call by reference mechanism in C#. In this case, also, both Num and Number variables going to share the same memory location, and hence changing the Number parameter value within the OUT method will also affect the Num parameter in the Main method.

```

using System;
namespace RefReturnsAndRefLocalsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Num = 1;
            //Calling the OUT Method using Pass By Reference Mechanism
            OUT(out Num);
            Console.WriteLine($"After the invocation of the {nameof(OUT)} method  
{nameof(Num)} = {Num}");

            Console.ReadLine();
        }
        //Declaring Method Parameter with out keyword
        //Here, Number will share the same memory address as Num
        static void OUT(out int Number)
        {
            //Changing Number Variable will change the Num Variable
            Number = 2;
        }
    }
}

```

```

    }
}

```

**Output:** After the invocation of the OUT method, Num = 2

When we run the application, the value returned from the OUT Method is shown, after the invocation of the OutMethod, Num = 2. C# 7.0, offers a shorter syntax for invoking the method with “out” parameters. The variable can be declared directly within the invocation.

## Example to Understand Out Variable Declare within Method Call in C#

Let us understand How to Declare Out Parameter within the Method call in C# with an example. Please have a look at the below example. Here, we are declaring the out variable number directly at the time of the method call.

```

using System;
namespace RefReturnsAndRefLocalsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Calling the OUT Method using Pass By Reference Mechanism
            OUT(out int Num);
            Console.WriteLine($"After the invocation of the {nameof(OUT)} method  
{nameof(Num)} = {Num}");

            Console.ReadLine();
        }
        //Declaring Method Parameter with out keyword
        //Here, Number will share the same memory address as Num
        static void OUT(out int Number)
        {
            //Changing Number Variable will change the Num Variable
            Number = 2;
        }
    }
}

```

**Output:** After the invocation of the OUT method, Num = 2

## Difference Between Ref and Out Parameters in C#

**Ref:**

1. The parameter or argument must be initialized first before it is passed to ref.

2. It is not required to assign or initialize the value of a parameter (which is passed by ref) before returning to the calling method.
3. Passing a parameter value by Ref is useful when the called method is also needed to modify the passed parameter.
4. It is not compulsory to initialize a parameter value before using it in a calling method.
5. The ref tells the compiler that the object is initialized before entering the function. so the data can be passed bi-directionally.
6. When you want to pass the value as well as you want to return the modified value then you need to use ref.

## Out:

1. It is not compulsory to initialize a parameter or argument before it is passed to an out.
2. A called method is required to assign or initialize a value of a parameter (which is passed to an out) before returning to the calling method.
3. Declaring a parameter to an out method is useful when multiple values need to be returned from a function or method.
4. A parameter value must be initialized within the calling method before its use.
5. The out tells the compiler that the object will be initialized inside the function, so the data is passed only in a unidirectional way i.e. from the called method to the caller method.
6. When you only want to return the value from the method then you need to use the out parameter.

**Note:** The point that you need to keep in mind is if a method just returns one value, then it's always better to use a return type instead of the out or ref modifier. OK. That's cool. Let's move to our main topic which is Ref Local and Ref Return in C# which was introduced as part of C# 7.

## Ref Local in C# 7

The **Ref Local in C#** is a new variable type that is used to store the references. It is mostly used in conjunction with Ref returns to store the reference in a local variable. That means Local variables now can also be declared with the ref modifier. If this is not clear at the moment, then don't worry, we will try to understand with some examples.

## Example to Understand Ref Local in C# 7

Let us understand Ref Local in C# 7 with an example. Please have a look at the below example. In the below example, first, we create an integer variable called no1 and initialized it with the value 1. Then we create another integer variable with the ref keyword with the name no2 and initialized it with the reference of no1 i.e. `ref int no2 = ref no1;` Now, the variable no2 references variable no1, and thus changing no2 changes no1 as well.

```
using System;
namespace RefReturnsAndRefLocalsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1 = 1;
```

```

        //Create a Local variable using ref keyword and store the reference
        another variable
        ref int Number2 = ref Number1;
        Number2 = 2;
        Console.WriteLine($"Local Variable {nameof(Number1)} After The Change: {Number1}");
        Console.ReadLine();
    }
}

```

**Output: Local Variable Number1 After The Change: 2**

## Ref Returns in C# 7

As a developer, you may be aware of the “ref” keyword and its behaviors. Before C# 7.0, the ref was only used to be passed as a parameter in a method. With C# 7.0, this constraint has been waived off and now you can return references from a method as well. This change is really adding flexibility to handle the scenarios when we want references to return in order to make an in-lined replacement. If this is not clear at the moment, then don't worry we will try to understand this concept with some examples.

### Example to Understand Ref Returns in C# 7

Let us understand Ref Returns in C# 7 with an example. Please have a look at the below example. In the below example, I am searching for an odd number inside an integer array and if it is not found throw an exception, the method is not returning it as the value but as a reference. So, we need to store that value that has been returned as a reference. To store it in a local variable, we can use the ‘ref’ keyword with local variables, known as ref locals in C#. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace RefReturnsAndRefLocalsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating an Integer Array
            int[] IntegerArray = { 2, 4, 62, 54, 33, 55, 59, 71, 92 };
            Console.WriteLine("Printing All Elements of IntegerArray");
            for (int i = 0; i < IntegerArray.Length; i++)
            {
                Console.Write($"{IntegerArray[i]}\t");
            }
            Console.WriteLine();
            //Creating an Instance of the Program class
            Program program = new Program();
            //Invoking the GetFirstOddNumber method with ref keyword
            //Storing the Result in a variable of ref type
            ref int OddNumber = ref program.GetFirstOddNumber(IntegerArray);

```

```

        //Printing the Odd Number returned by GetFirstOddNumber method
        Console.WriteLine($"First Odd Number Returned By GetFirstOddNumber
{OddNumber}");
        //Changing the oddNum value as it is a reference variable,
        //it will also change the same in the actual Integer Array
        OddNumber = 35;
        Console.WriteLine("Changing First Odd Number Value to 35");
        //Printing all the Elements of the IntegerArray
        Console.WriteLine("Printing All Elements of IntegerArray After Ch
Odd Number");
        for (int i = 0; i < IntegerArray.Length; i++)
        {
            Console.Write($"{IntegerArray[i]}\t");

            Console.ReadKey();
        }
        //Making the Method return type as ref int to return the reference of
number
        public ref int GetFirstOddNumber(int[] numbers)
        {
            for (int i = 0; i < numbers.Length; i++)
            {
                if (numbers[i] % 2 == 1)
                {
                    //Returning the Reference of the Number
                    return ref numbers[i];
                }
            }
            throw new Exception("Odd Number Not Found");
        }
    }
}

```

If you print “OddNumber” the first time, then it will print 33 but after that, I have re-assigned its value and set “OddNumber=35”. Then iterating the array and printing elements of the array and you can see that whatever I have done, a modification for “OddNumber” from outside is also reflected inside the array and the internal value has been modified from 33 to 35. So, when you run the above code, you will get the following output.

```

Printing All Elements of IntegerArray
2      4      62      54      33      55      59      71      92
First Odd Number Returned By GetFirstOddNumber Method: 33
Changing First Odd Number Value to 35
Printing All Elements of IntegerArray After Changing First Odd Number
2      4      62      54      35      55      59      71      92

```

In the next article, I am going to discuss the [Generalized Async Return Types in C#](#) with Examples. Here, in this article, I try to explain **How to Use Ref Returns and Ref Locals in C# 7** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Ref Returns and Ref Locals in C# 7 with Examples article.